

Laboratorio 2

Punto 1

```
-- Retorna la longitud de una lista
milength :: [a] -> Int
milength [] = 0
milength (_:xs) = 1 + milength xs
-- Retorna el último elemento de una lista
milast :: [a] -> a
milast [] = error "Lista vacía"
milast [x] = x
milast (_:xs) = milast xs
-- Duplica cada elemento de una lista de números
duplicar :: [Int] -> [Int]
duplicar = map (* 2)
-- Eliminar los elementos pares de una lista
eliminarPares :: [Int] -> [Int]
eliminarPares [] = []
eliminarPares (x:xs)
  | odd x = x : eliminarPares xs
  | otherwise = eliminarPares xs
```

```
milength [1, 2, 3, 4, 5]
duplicar [1, 2, 3, 4]
eliminarPares [1, 2, 3, 4, 5, 6]
```

```
5
[2,4,6,8]
[1,3,5]
```

```
milength' :: [a] -> Int
milength' = foldr (\_ acc -> acc + 1) 0
milast' :: [a] -> a
milast' = foldl1 (\_ x -> x)
duplicar' :: [Int] -> [Int]
duplicar' = map (* 2)
eliminarPares' :: [Int] -> [Int]
eliminarPares' = filter odd
-- Prueba
duplicar [1, 2, 3, 4]
```

```
[2,4,6,8]
```

Punto 2

```
-- 1.a. Obtener los cuadrados de los primeros 10 números
cuadrados :: [Int]
cuadrados = [x^2 | x <- [1..10]]

-- 1.b. Filtrar los divisibles por 3 entre 1 y 30
divisiblesPor3 :: [Int]
divisiblesPor3 = [x | x <- [1..30], x `mod` 3 == 0]

-- 1.c. Generar lista de pares (x, y) donde x < y y ambos estén en [1..5]
paresOrdenados :: [(Int, Int)]
paresOrdenados = [(x, y) | x <- [1..5], y <- [1..5], x < y]
```

```
cuadrados
divisiblesPor3
paresOrdenados
```

```
[1,4,9,16,25,36,49,64,81,100]
[3,6,9,12,15,18,21,24,27,30]
[(1,2),(1,3),(1,4),(1,5),(2,3),(2,4),(2,5),(3,4),(3,5),(4,5)]
```

```
-- 2. Devuelve los divisores de un número
divisores :: Int -> [Int]
divisores n = [x | x <- [1..n], n `mod` x == 0]
```

```
divisores 28
```

```
[1,2,4,7,14,28]
```

```
-- 3. Devuelve los divisores propios de un número
divisoresPropios :: Int -> [Int]
divisoresPropios n = [x | x <- [1..n-1], n `mod` x == 0]
esPerfecto :: Int -> Bool
esPerfecto n = sum (divisoresPropios n) == n
```

```
divisoresPropios 28
```

```
[1,2,4,7,14]
```

Punto 3

```
-- 1. Reescribe Las siguientes funciones sin recursión explícita[span_2](start_span)[span_2](end_span)[span_3](start_span)[span_3](end_span)
-- a. Suma de todos Los elementos[span_4](start_span)[span_4](end_span)[span_5](start_span)[span_5](end_span)
sumalista :: [Int] -> Int
sumalista = sum
-- b. Contar cuántos elementos son mayores que 10[span_6](start_span)[span_6](end_span)[span_7](start_span)[span_7](end_span)
mayoresQue10 :: [Int] -> Int
mayoresQue10 xs = length (filter (> 10) xs)
-- c. Calcular el producto de una lista de números[span_8](start_span)[span_8](end_span)[span_9](start_span)[span_9](end_span)
productolista :: [Int] -> Int
productolista = product
-- d. Convierte una lista de grados Celsius a Fahrenheit[span_10](start_span)[span_10](end_span)[span_11](start_span)[span_11](end_span)
celsiusAFahrenheit :: [Double] -> [Double]
celsiusAFahrenheit = map (\c -> c * 9 / 5 + 32)
-- e. Suma 5 a cada número impar de una lista y deja igual los pares[span_12](start_span)[span_12](end_span)[span_13](start_span)[span_13](end_span)
ajustarImpares :: [Int] -> [Int]
ajustarImpares = map (\x -> if odd x then x + 5 else x)
```

```
-- a. Suma de todos Los elementos
sumalista [1, 2, 3, 4, 5]
-- b. Contar mayores que 10
mayoresQue10 [5, 12, 8, 20, 3, 15]
-- c. Producto de una lista
productolista [1, 2, 3, 4]
-- d. Grados Celsius a Fahrenheit
celsiusAFahrenheit [0.0, 10.0, 20.0]
-- e. Sumar 5 a impares y dejar pares igual
ajustarImpares [1, 2, 3, 4, 5]
```

```
-- 2. Define Las siguientes funciones usando filter[span_14](start_span)[span_14](end_span)[span_15](start_span)[span_15](end_span)
-- a. Devuelve solo los nombres con más de 5 letras[span_16](start_span)[span_16](end_span)[span_17](start_span)[span_17](end_span)
nombresLargos :: [String] -> [String]
nombresLargos = filter (\s -> length s > 5)
-- b. Filtra los números negativos de una lista[span_18](start_span)[span_18](end_span)[span_19](start_span)[span_19](end_span)
soloNegativos :: [Int] -> [Int]
soloNegativos = filter (< 0)
-- c. De una lista de edades, obtiene las que estén entre 18 y 25 años[span_20](start_span)[span_20](end_span)[span_21](start_span)[span_21](end_span)
edadesUniversitarios :: [Int] -> [Int]
edadesUniversitarios = filter (\e -> e >= 18 && e <= 25)
```

```
-- a. Nombres con más de 5 letras
nombresLargos ["Ana", "Santiago", "Luis", "Carlos", "Fernando"]
-- b. Filtra los números negativos
soloNegativos [5, -3, 8, -10, 0, -1]
-- c. Edades entre 18 y 25 años
edadesUniversitarios [15, 18, 20, 22, 25, 30]
```

```
-- 3. Define Las siguientes funciones usando filter y map[span_22](start_span)[span_22](end_span)
-- a. Sumar IVA (19%) solo a los valores mayores o iguales a 1000[span_23](start_span)[span_23](end_span)
agregarIVA :: [Double] -> [Double]
agregarIVA xs = map (* 1.19) (filter (>= 1000) xs)
-- b. Obtener de palabras solo las que comienzan con vocal y devolver su Longitud[span_24](start_span)[span_24](end_span)
longitudesVocales :: [String] -> [Int]
longitudesVocales palabras = map length (filter esVocalInicial palabras)
  where
    esVocalInicial p = not (null p) && toLower (head p) `elem` "aeiou"
-- c. Devuelve Los cuadrados de Los números pares[span_25](start_span)[span_25](end_span)
cuadradosPares :: [Int] -> [Int]
cuadradosPares xs = map (^ 2) (filter even xs)
```

```
-- a. Sumar IVA (19%) a valores >= 1000
agregarIVA [500.0, 1000.0, 2000.0]
-- b. Longitudes de palabras que comienzan con vocal
longitudesVocales ["Hola", "avion", "elefante", "sol"]
-- c. Cuadrados de Los números pares
cuadradosPares [1, 2, 3, 4, 5, 6]
```

```
-- 4. Define Las siguientes funciones usando any o all[span_26](start_span)[span_26](end_span)
-- a. Verifica si algún número en La Lista es negativo[span_27](start_span)[span_27](end_span)
hayNegativo :: [Int] -> Bool
hayNegativo = any (< 0)
-- b. Verifica si todos Los números son pares[span_28](start_span)[span_28](end_span)
todosPares :: [Int] -> Bool
todosPares = all even
-- c. Comprueba si todos Los nombres empiezan con mayúscula[span_29](start_span)[span_29](end_span)
nombresCorrectos :: [String] -> Bool
nombresCorrectos = all (\n -> not (null n) && isUpper (head n))
```

```
-- a. Verifica si algún número es negativo
hayNegativo [1, 2, -3, 4]
-- b. Verifica si todos Los números son pares
todosPares [2, 4, 6, 8]
-- c. Comprueba si todos Los nombres empiezan con mayúscula
nombresCorrectos ["Ana", "Pedro", "Luis"]
```

```
-- 5. Define Las siguientes funciones usando takeWhile o dropWhile[span_30](start_span)[span_30](end_span)
-- a. Toma Los elementos menores que 100 de una Lista ordenada[span_31](start_span)[span_31](end_span)
menoresQue100 :: [Int] -> [Int]
menoresQue100 = takeWhile (< 100)
-- b. Quita Los ceros iniciales de una Lista[span_32](start_span)[span_32](end_span)
sinCerosIniciales :: [Int] -> [Int]
sinCerosIniciales = dropWhile (== 0)
```

```
-- a. Toma elementos menores que 100 de una Lista ordenada
menoresQue100 [10, 50, 99, 105, 200]
-- b. Quita Los ceros iniciales
sinCerosIniciales [0, 0, 0, 5, 1, 0, 2]
```